

Hospital Management System

An Object-Oriented Programming Implementation

Student: Rhitam Biswas

Course: Object-Oriented Programming

Semester: Spring 2026



Problem Statement



Manual Systems

Hospitals rely on paper-based tracking, causing treatment delays and data entry errors



Critical Response

Life-threatening cases require immediate attention



Patient Handling

Inefficient patient management leads to longer wait times and reduced care quality

Solution: A Java-based system that prioritizes critical patients and assigns appropriate treatment pathways dynamically.

Project Overview



Console-Based Application

Java application that processes patient data through user input and a decision-making.

Core Functionality

- Collect patient details: name, disease, priority level
- Assign available doctor
- Automatically determine treatment type
- Handle emergency and normal cases differently

System Workflow

01

Data Collection

User inputs patient name, disease type, and priority level (critical or normal)

03

Patient Object Creation

Instantiates Patient object storing all collected attributes

05

Treatment Decision

If critical → emergency surgery. If normal → medicine or first aid

02

Doctor Assignment

User selects doctor

04

Condition Analysis

Doctor object evaluates patient condition and priority level

06

Treatment Execution

Appropriate treatment object created and treat() method invoked

Class Structure & Design



Patient Class

Stores patient data: name, disease, priority level. Encapsulates private variables with getter/setter methods.



Doctor Class

Decision-making component that analyzes patient condition and assigns appropriate treatment type.



Treatment Base Class

Abstract parent class defining common treatment interface and methods.



FirstAid Subclass

Minor injury treatment with basic care procedures.



Surgery Subclass

Emergency surgical procedures for critical conditions.



Medicine Subclass

Prescription medication for fever and non-critical illnesses.

UML Diagram & Relationships



Patient

Holds data attributes



Doctor

Analyzes & decides



Treatment

Abstract parent

FirstAid

Minor injury care

Surgery

Emergency operations

Medicine

Prescription drugs

Inheritance: FirstAid, Surgery, and Medicine inherit from Treatment base class.

Polymorphism: Dynamic method calls to treat() execute appropriate treatment type.

Key Features & OOP Concepts



Priority-Based Treatment

Critical patients receive immediate emergency surgery while normal cases get medicine or first aid



Condition-Based Logic

System diagnoses based on symptoms: fever → medicine, injury → first aid, heart issues → surgery



Doctors

TPatient can choose the doctor



Continuous Processing

Loop allows unlimited patient entries until user terminates program



Encapsulation

Private variables in Patient class ensure data security

Inheritance

Treatment → subclasses create hierarchy

Polymorphism

Dynamic treat() method calls execute appropriate treatment

Abstraction

Common Treatment class defines interface

Implementation Highlights

Exception Handling

Try-catch blocks in main method handle invalid doctor selection and data type errors, preventing program crashes

Switch-Case Logic

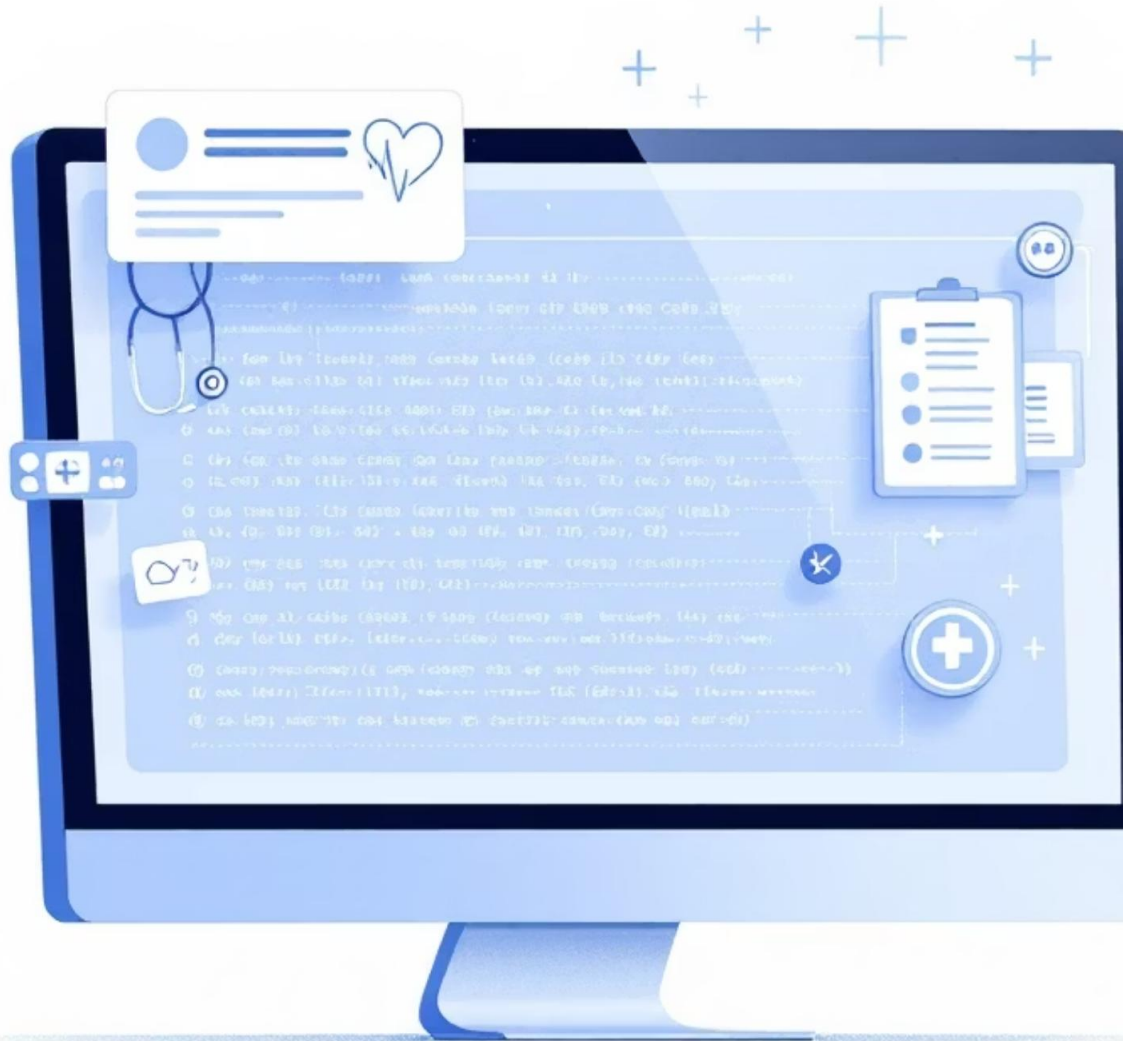
Doctor selection implemented using switch-case for efficient branching

If-Else Conditions

Priority handling uses nested if-else for critical vs normal patient routing

Dynamic Objects

Treatment type objects created at runtime based on patient condition



Testing & Sample Output

Test Case 1

Patient: Rhitam (Critical)

Disease: Heart Attack

Result: Emergency Surgery

Test Case 2

Patient: Arijit (Normal)

Disease: Fever

Result: Medicine

Test Case 3

Patient: Suranjan (Normal)

Disease: Injury

Result: First Aid

Edge Case Handling

- Invalid doctor input triggers error message
- Unknown disease defaults to general treatment
- Continuous loop allows multiple patient entries

Console Output

Step-by-step patient processing displayed with treatment decisions and doctor assignments

Conclusion & Future Scope

Project Success

Successfully implements OOP concepts including encapsulation, inheritance, polymorphism, and abstraction through a practical hospital management system.

System efficiently processes patients with priority-based routing and automated treatment assignment.

